

树的直径(diameter.cpp)

【问题描述】

小 Y 刚学完如何求解树的直径，为了巩固自己对树的理解，小 Y 画了一颗结点数为 N 的超级大的树，突然，他想到了老师上课讲的树的直径的问题。

他想知道，如果删去 K 条边，把树分成 $K+1$ 个联通块，所有联通块中最长的直径长度最小是多少

此处树的直径长度指树上最长简单路径的长度

为了防止递归过深爆栈，加入编译命令 `-Wl,--stack=10000000`

【输入格式】

第一行两个整数 N, K , 分别表示树的大小和删除的边数

后面 $N-1$ 行，每行两个数 u, v 表示树的一条边

【输出格式】

一行一个整数，表示所有删去 K 条边的情况，所有联通块的最长直径的最小值

【输入样例】

```
6 2
1 2
1 3
1 4
2 5
3 6
```

【输出样例】

```
1
```

【样例解释】

删去 $(1,2), (1,3)$ 两条边后，剩下的部分均为两个点连在一起，其中最长路径长度均为 1.

【数据范围】

对于 30% 的数据, $N, K \leq 30$;

对于 50% 的数据, $N, K \leq 50000$;

对于额外 10% 的数据, $N \leq 400000, K=0$,

对于额外 10% 的数据, $N \leq 400000$, 保证数据为一条链

对于 100%的数据, $N \leq 400000$, $K \leq N-1$.

数组变换(array.cpp)

【问题描述】

你的面前有一个长度为 n 的数组。初始时，第 i 个位置的元素为 i （即所有位置上的元素依次为从 1 到 n 的正整数）。

接下来，你需要帮我依次完成 m 个操作。每个操作可能是：

$A\ p\ q$ ，表示你需要修改数组中的所有元素。具体地，第 i 个位置的元素被修改成 $p \times i + q$ 。

$B\ x\ y$ ，表示你需要修改数组中的单个元素。具体地，第 x 个位置的元素被修改成 y 。

其中上述 p, q, x, y 均为整数，每次操作结束后，你需要输出数组中所有元素的和。

【输入格式】

接下来 m 行，每行依次表示一个操作，格式为 $A\ p\ q$ 或者 $B\ x\ y$ ，意义如上所述。

【输出格式】

输出共 m 行，每行表示每个操作结束后，数组中所有数的和。

【输入样例】

```
5 4
B 1 5
A -1 0
A 1 2
A 2 -5
```

【输出样例】

```
19
-15
25
5
```

【样例解释】

第 1 次操作结束后，数组变为{5, 2, 3, 4, 5};

第 2 次操作结束后，数组变为{-1, -2, -3, -4, -5};

第 3 次操作结束后，数组变为{3, 4, 5, 6, 7};

第 4 次操作结束后，数组变为{-3, -1, 1, 3, 5}。

【数据范围】

对于 30%的数据， $n \leq 100$;

对于额外 20%的数据， $n \leq 5e7, m \leq 5e5$, 所有操作均为 B 类型

对于额外 20%的数据， $n \leq 5e7, m \leq 5e5$, 所有操作均为 A 类型

对于 100%的数据， $n \leq 5e7, m \leq 5e5, |p| \leq 1000, |q|, |y| \leq 1e9,$
 $1 \leq x \leq n$

时间复杂度(complexity.cpp)

【问题描述】

小明正在学习一种新的编程语言 A++，刚学会循环语句的他激动地写了好多程序并给出了他自己算出的时间复杂度，可他的编程老师实在不想一个一个检查小明的程序，于是你的机会来啦！下面请你编写程序来判断小明对他的每个程序给出的时间复杂度是否正确。

A++语言的循环结构如下：

```
F i x y
    循环体
E
```

其中 `F i x y` 表示新建变量 `i`（变量 `i` 不可与未被销毁的变量重名）并初始化为 `x`，然后判断 `i` 和 `y` 的大小关系，若 `i` 小于等于 `y` 则进入循环，否则不进入。每次循环结束后 `i` 都会被修改成 `i+1`，一旦 `i` 大于 `y` 终止循环。

`x` 和 `y` 可以是正整数（`x` 和 `y` 的大小关系不定）或变量 `n`。`n` 是一个表示数据规模的变量，在时间复杂度计算中需保留该变量而不能将其视为常数，该数远大于 100。

“E”表示循环体结束。循环体结束时，这个循环体新建的变量也被销毁。

注：本题中为了书写方便，在描述复杂度时，使用大写英文字母“O”表示通常意义下“ Θ ”的概念。

【输入格式】

输入文件第一行一个正整数 `t`，表示有 `t` ($t \leq 10$) 个程序需要计算时间复杂度。每个程序我们只需抽取其中 `F i x y` 和 `E` 即可计算时间复杂度。注意：循环结构允许嵌套。

接下来每个程序的第一行包含一个正整数 `L` 和一个字符串，`L` 代表程序行数，字符串表示这个程序的复杂度，`O(1)` 表示常数复杂度，`O( $n^w$ )` 表示复杂度为 n^w ，其中 `w` 是一个小于 100 的正整数（输入中不包含引号），输入保证复杂度只有 `O(1)` 和 `O( $n^w$ )` 两种类型。

接下来 `L` 行代表程序中循环结构中的 `F i x y` 或者 `E`。程序行若以 `F` 开头，表示进入一个循环，之后有空格分离的三个字符（串）`i x y`，其中 `i` 是一个小写字母（保证不为 `n`），表示新建的变量名，`x` 和 `y` 可能

是正整数或 n ，已知若为正整数则一定小于 100。

程序行若以 E 开头，则表示循环体结束。

【输出格式】

输出文件共 t 行，对应输入的 t 个程序，每行输出 Yes 或 No 或者 ERR（输出中不包含引号），若程序实际复杂度与输入给出的复杂度一致则输出 Yes，不一致则输出 No，若程序有语法错误（其中语法错误只有：① F 和 E 不匹配 ②新建的变量与已经存在但未被销毁的变量重复两种情况），则输出 ERR。

注意：即使在程序不会执行的循环体中出现了语法错误也会编译错误，要输出 ERR。

【输入样例】

```
8
2 O(1)
F i 1 1
E
2 O(n^1)
F x 1 n
E
1 O(1)
F x 1 n
4 O(n^2)
F x 5 n
F y 10 n
E
E
4 O(n^2)
F x 9 n
E
F y 2 n
E
4 O(n^1)
F x 9 n
F y n 4
```

E
E
4 O(1)
F y n 4
F x 9 n
E
E
4 O(n^2)
F x 1 n
F x 1 10
E
E

【输出样例】

Yes
Yes
ERR
Yes
No
Yes
Yes
ERR

【样例解释】

第一个程序 i 从 1 到 1 是常数复杂度。

第二个程序 x 从 1 到 n 是 n 的一次方的复杂度。

第三个程序有一个 F 开启循环却没有 E 结束，语法错误。

第四个程序二重循环，n 的平方的复杂度。

第五个程序两个一重循环，n 的一次方的复杂度。

第六个程序第一重循环正常，但第二重循环开始即终止（因为 n 远大于 100，100 大于 4）。

第七个程序第一重循环无法进入，故为常数复杂度。

第八个程序第二重循环中的变量 x 与第一重循环中的变量重复，出现语法错误②，输出 ERR。

【数据范围】

对于 30%的数据：不存在语法错误，数据保证小明给出的每个程序的前 $L/2$ 行一定为以 F 开头的语句，第 $L/2+1$ 行至第 L 行一定为以 E 开头的语句， $L \leq 10$ ，若 x 、 y 均为整数， x 一定小于 y ，且只有 y 有可能为 n 。

对于 50%的数据：不存在语法错误， $L \leq 100$ ，且若 x 、 y 均为整数， x 一定小于 y ，且只有 y 有可能为 n 。

对于 70%的数据：不存在语法错误， $L \leq 100$ 。

对于 100%的数据： $L \leq 100$ 。